

PYTHAI / DELTAVERSE Production Architecture: DAIO + Custom Algorand DEX + x402 + Mirrortokens

Author: prepared for Gregory Magnusson (codephreak / Professor Codephreak) **Status:** v1 specification, mainnet-target, Apache-2.0 **Standard:** cypherpunk2048 — terse, plain-text, modular, no admin keys post-deploy, no upgradeable proxies

TL;DR

- **The system is a dual-chain BONAFIDE topology:** Algorand carries the constitutional layer (DAIO of 13 ASA seats + the custom Tinyman-v2-derived DEX `Forum` + the `Genius`-locked liquidity NFT + x402-on-Algorand via Parsec); Ethereum mainnet carries the economic anchor layer (xERC20 mirrortokens with an `XLockbox`, the LayerZero V2 + CCIP + Wormhole NTT bridge fabric, and the EVM-side proof-of-voting bridge). The three live services consume both rails through a single `allchain.html` registry.
 - **The 100-day automatic timelock with a 90→100-day 10-day redemption window is the core anti-rug invariant:** at the first `bootstrap`+`add_initial_liquidity` atomic group, the pool's LP ASA is forcibly clawback-transferred into the `Genius` lock contract and a non-transferable **Tessera-LP ASA NFT** (the certificate of redemption) is minted to the depositor. No address — not even the deployer — can withdraw before day 90 or after day 100; on day 101 the lock auto-rolls to a fresh 100-day epoch. **A 100-day initial lock sits just above the explicit DEXTools/Phemex <30-day "stop sign" line but well below the credibility baseline of 6 months;** the architecture compensates with a permanent auto-roll, non-renounceable, no-admin-key design that makes "wait for the lock to expire, then rug" structurally impossible.
 - **Implementation language stack is fixed:** Algorand contracts = Algopy/PuyaPy under AlgoKit 3.0 with `algorand-python-testing` for unit tests; EVM contracts = Solidity with Foundry forge-test against an Ethereum mainnet fork at <0.2 gwei; orchestration in OpenBSD vmm guests running Podman rootless; UI is minimal reactive (NiceGUI server-side, no SPA). x402 endpoints use the existing PYTHAI-authored PHP library on the mindX side and Coinbase-canonical TypeScript/Python SDKs on the agent side.
-

Key Findings

1. **Tinyman V2 is the correct fork base.** Its architecture is one stateful Validator App + a deterministic Pool LogicSig template per ASA pair (Tinyman Docs / `tinyman-amm-contracts-v2`). Pool addresses are derived from `(asset1_id, asset2_id,`

`validator_app_id`), the Pool Token ASA is created by the app via inner transaction during `bootstrap`, and the default swap fee is 0.3% with a 1/6 protocol-fee fraction (i.e. 0.05% of total input) that **accumulates in both pool reserve assets** (not in LP tokens — Tinyman V2 changed this from V1) and is claimable by any caller via inner transfers to a fee collector. The Tinyman source is Tealish; rewriting to Algopy/PuyaPy is straightforward because Puya targets the same AVM features and AlgoKit 3.0 ships first-class `algorand-python-testing`. (GitHub + 4)

2. **The "automatic on first-pairing" timelock has no clean Algorand precedent.** Tinyman LP tokens are freely transferable ASAs; Tinyman's own docs confirm "no locking or movement of these tokens required" for farming. Pact is similarly open. The closest design analog is the Solana pump.fun auto-burn-on-graduation pattern (which migrated from Raydium to PumpSwap in late March 2025; LPs are now sent to a dead address at the ~\$75k market-cap graduation), but pump.fun *destroys* the LP rather than time-locking it. The PYTHAI design — `Genius` intercepting LP minting via an atomic-group invariant on `add_initial_liquidity`, plus a `Tessera-LP` ASA NFT as the redemption certificate — is therefore **net-new on Algorand** and should be cited as such in BONAFIDE v1. (Tinyman + 2)
3. **xERC20 (ERC-7281) is the right mirrortoken standard for the EVM side, paired with Wormhole NTT on the Algorand side.** xERC20 gives the token issuer sovereign mint/burn whitelisting per-bridge with rate limits and an `xERC20Lockbox` for the canonical chain; Wormhole NTT — specifically the Folks Finance Algorand port (PuyaPy implementation at `Folks-Finance/algorand-ntt-contracts`), with a private audit dated October 2025 and a public Immunefi competition with a \$30,000 reward pool that closed November 20, 2025) — gives ASA-native cross-chain minting without wrapped-asset duplication. LayerZero V2 (DVN security stack, X-of-Y-of-N quorum on Ultra Light Node MessageLib) and Chainlink CCIP (dual-DON + separate Risk Management Network, CCT standard) are wired in as **redundant verifier bridges** behind the xERC20 facade, not as competing bridges. This is the Pontifex pattern: many bridges, one sovereign token contract on each chain. (Xerc20 + 4)
4. **No published artifact ties "BONAFIDE", "Genius", "Tabularium", "Fides", "Sponsio Pactum", "Censura", "Senatus", "Tessera", or "Curia" to codephreak today.** Exhaustive search of GitHub (`pythaiml`), `Professor-Codephreak`, `deltav-deltaverse`, the Magnusson Medium, and Latin-keyword combinations returned nothing. This document therefore serves as the **first canonical publication** of the BONAFIDE naming substrate; the architecture below establishes each contract's role definitively.
5. **The DAIO treasury/13 valuation must use Pyth Network on Algorand for the float oracle** — not a custom feed. Pyth maintains an Algorand target chain and provides sub-second pulled-on-demand price updates; this is materially cheaper and more secure than building a bespoke oracle. The `Tabularium` (treasury ledger app) reads Pyth prices per asset in the treasury, sums to a USD figure, and publishes `seat_value = treasury_usd / 13` to global state on every governance-relevant transaction. Seat NFTs (Senatus seats) do not change

their ASA properties when value floats — only the on-chain accounting (`seat_value`) does.

([GitHub](#))

6. **x402 on Algorand requires a new scheme**, because the canonical Coinbase facilitator only ships (`exact_evm`) and (`exact_svm`). PYTHAI must publish (`exact_avm`) (the Algorand scheme) into the x402 (`specs/schemes`) directory, modeled on the (`OrbytLabz/x402python`) Solana port pattern. Parsec (the user's wallet) signs the payment payload locally; the AVM-side facilitator verifies the signature and submits an atomic-group payment-plus-application-call. Because Algorand has no EIP-3009 equivalent, the scheme uses ARC-60 arbitrary signing with a domain-bound (`authenticatorData`) to prevent cross-domain replay.
7. **mindX is the highest-value x402 surface in the ecosystem** — every reasoning call is naturally pay-per-inference; the PHP library on (`mindx.pythai.net`) returns HTTP 402 with (`PAYMENT-REQUIRED`) headers, the agent's Parsec wallet signs an (`exact_avm`) scheme payload, and the response 200 carries an (`X-PAYMENT-RESPONSE`) confirmation. AgenticPlace acts as the agent registry and routes; BANKON is the identity binder that maps Algorand ARC-52 HD addresses to the agent's marketplace persona. ([Cloudflare](#)) ([Coinbase](#))

Details

Section 1 — Custom Algorand DEX: ([Forum](#)) (Tinyman-v2-derived with Liquidity Timelock)

1.1 Contract topology (Algopy/PuyaPy)

```
forum/
  validator_app.py          # ARC4Contract - replaces Tinyman Validator App
  pool_logicsig.py.tpl     # parametric stateless contract (asset1, asset2,
  validator_app_id, GENIUS_APP_ID)
  genius_lock.py           # ARC4Contract - timelock + Tessera-LP issuance
  tessera_lp_nft.py        # ASA NFT factory: per-pool, per-depositor redemption
  certificate
  curia_router.py          # multi-hop swap router (read-only state aggregator on
  top of Validator)
  fides_oracle_adapter.py  # Pyth reader for USD-denominated treasury accounting
```

(`validator_app`) keeps the Tinyman V2 entry points ((`bootstrap`), (`add_initial_liquidity`), (`add_subsequent_liquidity`), (`remove_liquidity`), (`swap`), (`flash_loan`), (`flash_swap`)) but with one breaking change: (`add_initial_liquidity`) is the only path that ever issues LP tokens, and it **MUST** atomically group with a call to (`genius_lock.lock_initial(...)`). If the Genius lock call is absent from the atomic group, the validator rejects the bootstrap. There is no off-by-one path.

1.2 The Liquidity Timelock state machine (Genius)

States (stored as a `UInt8` enum in global state, keyed by `pool_address`):

```
0 UNINITIALIZED      no liquidity ever paired
1 LOCKED_EPOCH       lock_start_ts ≤ now < lock_start_ts + 90 days
2 REDEMPTION_WINDOW  lock_start_ts + 90 days ≤ now < lock_start_ts + 100 days
3 AUTO_ROLLED        now ≥ lock_start_ts + 100 days, no redemption taken → state
                    -> 1 with lock_start_ts := now
```

Transitions:

- `UNINITIALIZED → LOCKED_EPOCH`: triggered atomically by the first `add_initial_liquidity` call. `lock_start_ts := Global.LatestTimestamp`. The newly minted Pool LP ASA is clawback-transferred (the LP ASA's `clawback` field is set to the Genius app account at creation) from the depositor into the Genius app account. A Tesseract-LP ARC-3 ASA NFT is inner-minted with:
 - `unit_name`: "TLP"
 - `name`: "Tesseract-LP {asset1}-{asset2} #{nonce}"
 - `decimals`: 0, `total`: 1 (NFT)
 - `manager`, `freeze`, `clawback`: zero address (immutable)
 - `reserve`: the Genius app account (holds metadata pointer)
 - URL: `ipfs://...` with the redemption manifest (depositor address, LP amount locked, `lock_start_ts`, `pool_address`) Sent to the depositor. The depositor can transfer the Tesseract-LP NFT freely (secondary market on AgenticPlace), and whoever holds it at redemption time can redeem.
- `LOCKED_EPOCH → REDEMPTION_WINDOW`: time-based, no transaction needed. The state derivation is purely a function of `Global.LatestTimestamp - lock_start_ts`; the `state()` ABI method is a read.
- `REDEMPTION_WINDOW → (redeemed)`: holder of Tesseract-LP submits an atomic group: (i) Tesseract-LP NFT axfer to Genius app, (ii) `genius_lock.redeem(pool_address)` app call. Genius verifies the NFT corresponds to the pool, axfers the underlying LP ASA back to the redeemer, then calls `validator_app.remove_liquidity` on behalf of the redeemer (inner transaction) returning the asset1+asset2 share. The Tesseract-LP NFT is then destroyed via inner `acfg destroy`.
- `REDEMPTION_WINDOW → AUTO_ROLLED → LOCKED_EPOCH`: at `now ≥ lock_start_ts + 100 days`, any caller's swap or LP operation on the pool implicitly triggers `genius_lock._maybe_roll(pool_address)`, which sets `lock_start_ts :=`

`Global.LatestTimestamp`) and leaves the LP ASA in the Genius account. No external action required.

Why this is provably rug-proof:

1. The LP ASA's `clawback` field is the Genius app account, set at creation in the same inner transaction that mints it. Algorand ASA fields are immutable in their `manager`/`clawback`/`freeze`/`reserve` configuration UNLESS those are set to the zero address ahead of time. Here `manager` of the LP ASA is also set to the Genius app — and Genius's approval program contains no `manager` reconfiguration path. The LP ASA is therefore permanently clawback-controlled by Genius logic, not by any human. `Algorand`
2. Genius has no admin key, no upgrade path (no `apsu` clear-state mutation; `extra_program_pages` budget consumed at deploy is the full final budget), and no path in approval logic that transfers LP out of itself before `state == REDEMPTION_WINDOW` of the corresponding pool.
3. The redemption window NFT is non-fungible per deposit (nonce-based naming), so even a partial redemption is one-shot per Tessera.
4. Even if every PYTHAI signer is compromised, the attacker cannot mint a fake Tessera-LP (only Genius's inner transaction at `add_initial_liquidity` mints them) and cannot reach the LP ASA without the genuine Tessera-LP arriving in the same atomic group.

Comparison to existing lockers:

Locker	Trust model	Auto-on-creation	NFT certificate	Lock duration
UNCX/Unicrypt	EVM, audited, locker contract is non-upgradable on V3	No — user chooses	Yes (lock NFT)	User-chosen, often 6–12 mo
Team Finance	EVM; on October 27, 2022 the v2→v3 <code>migrate()</code> function was exploited for \$14.5M USD because it failed to validate that the user's locked token matched the token being migrated, <code>Neptunemutual</code> draining WETH, CAW, USDC, and TSUKA pools <code>ImmuneBytes</code>	No	Lock receipt	User-chosen
PinkLock (PinkSale)	EVM, tied to PinkSale launchpad	Optional via launchpad	No	User-chosen
pump.fun (Solana)	LP burned , not locked; since late March 2025 burns happen at PumpSwap (not Raydium) graduation at ~\$75k mcap <code>Smithii</code>	Yes (on graduation)	None (burn)	Permanent
Forum/Genius (PYTHAI)	Algorand, no admin key, immutable LP ASA clawback	Yes — at first pairing, no user choice	Yes (Tessera-LP ASA NFT)	100-day auto-rolling epoch with 10-day window

Pitfalls surfaced and mitigations:

- Pitfall A: 100 days is short by industry baselines.** DEXTools (2026 guide) flags "Locks under 30 days are major red flags" and sets "Minimum 6 months: For small-cap tokens, this is the bare minimum"; Phemex states "locks under 30 days are a warning sign and no lock at all is a stop sign." The PYTHAI 100-day lock clears the explicit <30-day red-flag line by a comfortable margin but falls below the 6-month credibility baseline. The mitigation is the **auto-roll**: there is no "after the lock expires" — at day 101 the lock is *already* in a fresh 100-day epoch unless the Tessera-LP holder explicitly redeemed in days 90–100. Document this clearly in user-facing copy or the design will be misread as a 100-day soft rug. `DEXTools + 2`
- Pitfall B: Tessera-LP secondary market = redemption-right concentration.** A whale can buy up Tessera-LP NFTs in days 0–89 and execute a coordinated `remove_liquidity` in the 10-day window. Mitigation: cap per-Tessera-LP redemption to a fraction of pool depth (e.g.,

max 5% per atomic group) enforced in the validator's `remove_liquidity` path when the caller is Genius.

- **Pitfall C: Tessera-LP holders may forget to redeem.** The 10-day window plus auto-roll means a forgetful holder simply gets another 90-day countdown — this is a feature, not a bug, but the AgenticPlace UI should surface "next window opens in N days" prominently.
- **Pitfall D: Clawback-equipped LP ASA can technically be frozen by Genius logic.** Set `freeze` to zero address at LP ASA creation to remove this attack surface entirely; only clawback (not freeze) is needed for the locking mechanic.

1.3 Foundry vs AlgoKit test strategy

Forum is Algorand-only; tests are AlgoKit/Algopy. The full matrix:

- `tests/test_genius_state_machine.py` — `algorand-python-testing` context tests every state transition with synthetic `any_application()` and time mocks. (Algorand)
- `tests/test_lp_asa_immutability.py` — asserts the LP ASA's `manager`/`clawback`/`freeze`/`reserve` fields cannot be reconfigured by any approval-program path.
- `tests/test_atomic_group_invariant.py` — asserts every `add_initial_liquidity` group with a missing `genius_lock.lock_initial` call fails.
- `tests/test_tessera_redemption.py` — covers day 89 (rejected), day 90 (accepted), day 100 (accepted), day 101 (rolled, rejected).
- LocalNet sandbox integration: `algokit localnet start` and `pytest -m integration` for end-to-end (Pera wallet-compatible path). (Medium)

EVM contracts (mirrortoken side, §2) get the Foundry treatment.

Section 2 — Cross-Chain Bridging and Mirrortoken System

2.1 Contract topology

EVM side (Ethereum mainnet, Foundry/Solidity):

```

contracts/
  XPythai.sol          # xERC20 (ERC-7281) sovereign bridged token, immutable
  XLockbox.sol         # ERC-7281 lockbox for the canonical ETH-side asset (if
there is one)
  PeggedFactory.sol    # CREATE2 deployer for new mirrortoken deployments per
chain
  bridges/
    LZv2Adapter.sol    # LayerZero V2 OApp adapter, security stack = [LZ Labs DVN
required, Google Cloud DVN required, Polyhedra zkDVN optional, threshold=1]
    CCIPAdapter.sol    # Chainlink CCIP receiver+sender, burn-and-mint via CCT
standard
    WormholeAdapter.sol# Wormhole NTT manager peer (calls Folks NTT manager on
Algorand side)
  governance/
    EVMVoteSink.sol    # receives state-proof attested vote results from Algorand
DAIO

```

Algorand side (Algopy):

```

algorand/
  ntt_manager.py       # forked from Folks-Finance/algorand-ntt-contracts, PuyaPy
  ntt_token.py         # ASA controller (locking-mode for the canonical PYTHAI
asset on Algorand)
  ntt_rate_limiter.py  # rate limit configured per inbound/outbound bridge
  transceiver_manager.py # generic message handler (Algorand-specific
generalization)
  state_proof_emitter.py # publishes governance vote roots so EVMVoteSink can
verify

```

2.2 Mirrortoken mint/burn flow (sovereign per-bridge limits)

On the EVM side, `XPythai.sol` implements the ERC-7281 minimal interface:

```

solidity

function mint(address user, uint256 amount) external; // only allowlisted bridges
function burn(address user, uint256 amount) external;
function setLimits(address bridge, uint256 mintLimit, uint256 burnLimit) external; /
function mintingMaxLimitOf(address bridge) external view returns (uint256);
function burningMaxLimitOf(address bridge) external view returns (uint256);

```


The three bridge adapters compete on price/finality at the application layer; clients pick which to use. xERC20 ensures the resulting tokens are fungible regardless of route — this is the entire point of ERC-7281 ("Transferrable across chains with no slippage", per xerc20.com). (Xerc20)

2.3 Tie-in to the DEX timelock NFT

Tessera-LP NFTs are **NOT** bridgeable. They are scope-bound to Algorand because the redemption call must touch Genius on Algorand. This is by design: bridging the redemption right would expand the attack surface unnecessarily. Users who want EVM-side exposure to PYTHAI liquidity instead hold xPYTHAI on Ethereum, which is backed by the Algorand-side canonical PYTHAI asset locked in the NTT manager (locking mode, per Folks Finance Algorand NTT docs).

2.4 Pontifex sovereignty topology (the openBDK 1-relayer + 3-validator pattern as Algorand-side fallback)

The Pontifex layer is the **emergency-only path** if all three bridges (LayerZero, CCIP, Wormhole) are simultaneously unavailable. It consists of a 1-relayer + 3-validator multisig that can move xPYTHAI between chains by signing an ARC-60-style payload with 3-of-3 validators. Validators are bound to specific Algorand and Ethereum addresses at deploy. This is operationally identical to the early Milkomeda Algorand bridge stargate pattern (multisignature wrapping), with the difference that Pontifex's authority is rate-limited to a small fraction of the rate-limited xERC20 budget per epoch, so even a 3-of-3 validator compromise cannot drain the system.

2.5 Known bridge attack vectors and defenses

- **Wormhole-style signature-bypass exploit:** in the February 2022 Solana incident, "the attacker exploited the use of a deprecated, insecure function `[[load_current_index]]` to bypass signature verification and steal \$326 million (120k wETH)" (Halborn). The root cause was *not* a guardian-set compromise but a deprecated-function exploit that had a fix committed to Wormhole's public GitHub but not yet deployed to mainnet. Defenses in this architecture: xERC20 per-bridge mint caps cap blast radius even if Wormhole or any one bridge is exploited; immutable, no-upgrade contracts eliminate the "fix committed but not deployed" failure mode. (Halborn) (NomosLabs)
- **Team-Finance-style migration vulnerability:** on October 27, 2022, "\$14.5M USD of tokens were exploited through the audited v2 to v3 migration function" — defended here by no upgrade path, no migration function, immutable contracts. (Crypto News)
- **Validator-set replay:** defended by Algorand state proofs (Falcon post-quantum signatures, 256-round compaction) feeding into the EVM verifier — i.e. don't trust the bridge, trust the

Algorand chain itself. (Algorand)

- **Rate-limit exhaustion DoS:** documented behavior per the Folks Finance ImmuneFi scope — "If the rate limit is exceeded, the transfer is only delayed from completing." Tolerable.

(ImmuneFi)

Section 3 — DAIO On-Chain Governance: 13-Seat Mint with Treasury Math

3.1 The BONAFIDE constitutional substrate (first canonical publication)

The following Latin-civic naming is canonical from this document forward:

Name	Role	Contract type
Senatus	The 13-seat ASA collection; one ASA per seat	Algorand ASA (ARC-19 mutable metadata for name only, immutable supply=1 per seat)
Tessera	Voting token per seat — proof-of-voting receipt	Algorand ASA NFT minted per vote
Tessera-LP	LP redemption certificate (see §1)	Algorand ASA NFT
Curia	The voting application — assembly hall	Algopy ARC4Contract
Tabularium	Treasury ledger + valuation oracle reader	Algopy ARC4Contract
BonaToken	Treasury-denominated unit (1 BonaToken = treasury_usd / 13 / 10 ⁶ smallest unit)	Algorand ASA
Sponsio Pactum	Proposal lifecycle contract (covenant binding)	Algopy ARC4Contract
Censura	Proposal moderation / kill-switch (3-of-7 Counsellors only, no individual veto)	Algopy ARC4Contract
Fides	Pyth oracle adapter, trust feed	Algopy ARC4Contract
Genius	Liquidity lock app (§1)	Algopy ARC4Contract
Forum	The DEX validator app (§1)	Algopy ARC4Contract

Roles in the cabinet pattern (1 Convener + 7 Counsellors + 5 Voting Citizens = 13 seats):

- **Seat 0 — Convener:** agenda-setter; can call `Sponsio Pactum.propose(...)` directly.
- **Seats 1-7 — Counsellors:** 3-of-7 quorum required for `Censura.veto()`; any 1 can second a proposal.
- **Seats 8-12 — Voting Citizens:** rotating community seats; vote-weighted equally with Counsellors on final passage.

A proposal passes if and only if `≥ 7 of 13 Tessera` votes are cast `YES` within the voting window AND no `Censura.veto()` has been recorded.

3.2 Seat valuation math (treasury / 13, floating)

`Tabularium` exposes:

```
python

@arc4.abimethod(readonly=True)
def seat_value_usd(self) -> arc4.UInt64:
    treasury_usd = self._sum_assets_usd_via_pyth() # iterates over treasury ASA list
    return arc4.UInt64(treasury_usd // 13)

@arc4.abimethod(readonly=True)
def treasury_breakdown(self) -> arc4.DynamicArray[AssetUsd]:
    ...
```

The valuation iterates over a fixed `treasury_assets` global state list (extensible only by passed governance proposal), queries `Fides` (the Pyth adapter) for each asset's USD price, multiplies by held balance, sums, and integer-divides by 13. There is no admin path to change the divisor.

Floating semantics: the ASA seat itself is unchanged when treasury value moves. What moves is the on-chain accounting figure `seat_value_usd` that's surfaced to voters and to AgenticPlace UI. If a proposal requires "`≥ 1 seat-equivalent of capital staked`", `Sponsio Pactum.propose()` reads `seat_value_usd` at submission time and locks the proposal to that snapshot.

3.3 Proof-of-voting

Each `Curia.cast_vote(proposal_id, ballot)` call:

1. Verifies caller holds (or is rekeyed to an account holding) one of the 13 Senatus ASAs.
2. Inner-mints a `Tessera` ASA NFT to that seat's address with metadata `((proposal_id, ballot, timestamp, voter_address))`. NFT is non-transferable (clawback = Curia, freeze = Curia, both immutable).
3. Records the vote in box storage keyed by `((proposal_id, seat_id))`.

- 4. Emits an `ARC-72-style` event for indexer consumption.

The Tessera NFTs are the on-chain proof-of-voting record — cryptographic, immutable, indexable.

3.4 Dual-chain DAIO topology

- **Algorand = constitutional layer:** Senatus, Curia, Sponsio Pactum, Censura, Tabularium all live here. This is where votes are cast and where the treasury accounting lives.
- **Ethereum = economic layer:** `EVMVoteSink.sol` consumes Algorand state proofs (the Falcon-signed 256-round compaction) and can execute treasury actions on the EVM side — paying out xPYTHAI to a grant recipient, calling `XPythai.setLimits` (during the un-renounced epoch), etc. The EVM side is purely subordinate; constitutional truth is Algorand.

The bridge for governance is **NOT** LayerZero/CCIP/Wormhole — those are for token/value flow. Governance uses Algorand state proofs directly, verified by a Falcon-signature verifier contract on Ethereum (the Algorand Foundation has reference implementations; see [medium.com/algorand state-proofs material](https://medium.com/algorand-state-proofs)). This eliminates intermediary trust on the path that matters most.

Section 4 — x402 Algorand Payment System via Parsec

4.1 New x402 scheme: `exact_avm`

Coinbase's canonical x402 schemes are `exact_evm` (EIP-3009 / Permit2) and `exact_svm` (Solana SPL with facilitator fee-payer pattern). Neither natively works on Algorand. PYTHAI publishes `exact_avm`:

- **PaymentRequirements** (as returned in HTTP 402 `PAYMENT-REQUIRED` header, base64 JSON):

```
json
```

```

{
  "scheme": "exact",
  "network": "algorand:mainnet-v1.0",
  "amount": "1000",
  "asset": "31566704",          // ASA ID for USDC on Algorand
  "payTo": "BANKON...58CHAR",
  "maxTimeoutSeconds": 60,
  "extra": {
    "facilitator": "https://x402-avm.pythai.net/v1",
    "scheme_variant": "exact_avm",
    "sig_scheme": "arc60"
  }
}

```

Note: CAIP-2 for Algorand is `algorand:mainnet-v1.0` (the genesis hash short form).

- **PaymentPayload** (in `PAYMENT-SIGNATURE` header from client):
 - An ARC-60-style signed `authenticatorData` over the canonical-JSON payment body
 - Plus an unsigned-but-prepared `axfer` transaction the facilitator can submit
 - Signed by the buyer's ARC-52 HD key (derivation path `m'/44'/283'/'<account>'/0/0`) per the [algorandfoundation/xHD-Wallet-API spec](#) — Algorand SLIP-44 coin type is 283)
- **Facilitator endpoints** (PHP, mirroring the Coinbase TypeScript reference):
 - `POST /verify` — re-derives the signature, confirms ASA balance + opt-in status, returns `{valid: true|false, reason}`. [GitHub](#)
 - `POST /settle` — submits the prepared atomic group, waits for confirmation, returns `{success, transaction, network, payer}`. [GitHub](#)

4.2 PYTHAI PHP x402 library (existing — first production PHP implementation)

The PHP library lives behind `mindx.pythai.net` and other PHP endpoints. Pseudocode for the seller middleware:

```
php
```

```

require_once 'x402.php';

$middleware = new X402\PaymentMiddleware([
    'GET /v1/reason' => [
        'accepts' => [[
            'scheme' => 'exact',
            'network' => 'algorand:mainnet-v1.0',
            'amount' => '5000', // 0.005 USDC per call
            'asset' => '31566704',
            'payTo' => $_ENV['BANKON_ADDR'],
        ]],
        'description' => 'mindX reasoning inference, per call',
    ],
]);

$middleware->handle(); // returns 402 or proceeds to handler

```

Because PHP shops dominate small-team API deployments, this library is a material first — and slots cleanly into the PYTHAI stack which already uses PHP for `aiapi.php` on `ai.pythai.net`.

4.3 Wiring x402 into the three services

- **AgenticPlace:** agents register an `acceptedSchemes` field including `exact_avm`. The marketplace's "hire this agent" button triggers a client-side x402 flow where the buyer's Parsec wallet auto-signs the first invocation. Agent-to-agent payments (one agent paying another for sub-task delegation) use the same scheme — no human in the loop.
- **mindX:** the reasoning API meters tokens per inference and replies 402 if the buyer's Parsec wallet's pre-authorization is exhausted. Streaming responses are paid in chunks via `upto` scheme (experimental in x402 v2) once that ships.
- **BANKON:** holds the canonical agent identity ↔ Algorand address binding. When a 402-bearing request arrives at any service, the service can look up `BANKON.resolve(agent_id) → ARC-52 address` to verify the signer matches the registered identity. BANKON acts as the directory layer between human-readable identity and on-chain address.

4.4 Parsec wallet, ARC-52 + state-proof awareness

Parsec's roadmap (per the requirements) layers two new capabilities:

- **ARC-52 HD wallet:** BIP32-Ed25519 derivation on the non-linear keyspace, Peikert's mode (more entropy in `zL`) per the `@algorandfoundation/xhd-wallet-api` library. Derivation paths: `m'/44'/283'/account'/change/index`. This lets one mnemonic derive every agent, marketplace listing, and BANKON identity. `npm` `npm`

- **State-proof quantum awareness:** Parsec ships with the Falcon signature verifier so it can independently validate Algorand state proofs received from any peer, removing intermediary trust on cross-chain governance reads.
-

Section 5 — Chain Mapping (`allchain.html`)

`allchain.html` was not fetchable (private page). The expected schema, used by all PYTHAI services for chain discovery, is:

```
json
```

```
{
  "version": "1.0.0",
  "updated": "2026-05-28T00:00:00Z",
  "chains": {
    "algorand:mainnet-v1.0": {
      "caip2": "algorand:mainnet-v1.0",
      "rpc": ["https://mainnet-api.algonode.cloud"],
      "indexer": ["https://mainnet-idx.algonode.cloud"],
      "explorer": "https://allo.info",
      "contracts": {
        "forum_validator_app": {"app_id": 0, "address": ""},
        "genius_lock": {"app_id": 0, "address": ""},
        "curia": {"app_id": 0, "address": ""},
        "tabularium": {"app_id": 0, "address": ""},
        "sponsio_pactum": {"app_id": 0, "address": ""},
        "censura": {"app_id": 0, "address": ""},
        "fides": {"app_id": 0, "address": ""},
        "ntt_manager": {"app_id": 0, "address": ""},
        "ntt_token": {"app_id": 0, "address": ""}
      },
      "asas": {
        "PYTHAI": {"asset_id": 0},
        "BONA": {"asset_id": 0},
        "USDC": {"asset_id": 31566704},
        "senatus_collection": [/* 13 ASA IDs */],
        "tessera_factory_app": 0,
        "tessera_lp_factory_app": 0
      },
      "facilitators": {
        "x402_avm": "https://x402-avm.pythai.net/v1"
      }
    },
    "eip155:1": {
      "caip2": "eip155:1",
      "rpc": ["https://ethereum-rpc.publicnode.com"],
      "explorer": "https://etherscan.io",
      "contracts": {
        "xpythai": {"address": "0x..."},
        "xlockbox": {"address": "0x..."},
        "lzv2_adapter": {"address": "0x..."},
        "ccip_adapter": {"address": "0x..."},
        "wormhole_adapter": {"address": "0x..."},
        "evm_vote_sink": {"address": "0x..."}
```



```

    }
  },
  "services": {
    "agenticplace": {"url": "https://agenticplace.pythai.net", "x402_schemes": ["ex
    "mindx": {"url": "https://mindx.pythai.net", "x402_schemes": ["ex
    "bankon": {"url": "https://bankon.pythai.net", "role": "identity"}
  }
}

```

Every deploy step (§7) appends/updates this file via a signed commit (deployer key, recorded on Algorand via a small notarization txn). Services fetch the file on boot and on `SIGHUP`.

Section 6 — Integration Wiring: The Three Services

6.1 AgenticPlace (agenticplace.pythai.net)

- **On-chain reads:** pulls `allchain.html` at boot, then queries Algorand indexer for: active proposals (`Sponsio Pactum`), open Tessera-LP redemption windows (`Genius`), and the current `seat_value_usd` (`Tabularium`).
- **On-chain writes:** triggers x402 payment flows for agent hires; mints AgenticPlace listing NFTs as ARC-19 ASAs (each listing is an ASA NFT, transferable as a secondary market).
- **DAIO integration:** any agent whose listing reaches a configurable popularity threshold can be proposed as a "Voting Citizen" candidate via `Sponsio Pactum`.
- **Tessera-LP integration:** Tessera-LP NFTs are tradeable on AgenticPlace as a first-class collection — pre-redemption-window trading is the secondary market.

6.2 mindX (mindx.pythai.net)

- **API surface** (RESTful, all routes 402-protected unless noted):

```

POST /v1/reason          x402-paid; submit a reasoning prompt, get a response
POST /v1/reason/stream   x402-paid via `upto`; SSE streaming response
GET /v1/agents/me        free; returns the calling agent's BANKON profile
POST /v1/agents/me/budget x402-paid via Permit2-style pre-auth; sets a wallet-
wide max spend
GET /v1/oracle/seat_value free; proxies Tabularium.seat_value_usd() for UI
GET /v1/oracle/treasury   free; proxies Tabularium.treasury_breakdown()
POST /v1/dao/proposal      x402-paid (gas-equivalent); submits a Sponsio Pactum
proposal on-chain
POST /v1/dao/vote          x402-paid (gas-equivalent); calls Curia.cast_vote
GET /v1/dex/pools          free; list Forum pools + Genius lock state + Tessera-
LP availability
POST /v1/dex/swap          x402-paid; routes through Curia router (multi-hop)
POST /v1/dex/redeem_tlp    x402-paid; redeems a Tessera-LP NFT in the 10-day
window

```

- **On-chain event flow:** mindX subscribes to Algorand indexer streams for: every Senatus vote, every Sponsio Pactum proposal, every Genius state transition, every Tessera-LP mint/redeem. These flow into PostgreSQL with pgvectorscale for the RAGE retrieval system (per the existing PYTHAI architecture). (GitHub)
- **Reasoning attribution:** each `/v1/reason` response includes an `X-MINDX-RECEIPT` header containing the ASA payment txID — this is the inference-was-paid attestation.

6.3 BANKON (bankon.pythai.net)

- **Identity binding:** `BANKON.resolve(agent_id) → ARC-52 address` and `BANKON.resolve(address) → agent_id` are the canonical lookup paths.
- **Token layer:** BANKON issues `BONA` ASA, a soulbound (clawback=BANKON, freeze=BANKON, both renounced post-bootstrap) reputation token earned via verified agent activity. BONA is non-transferable; it represents identity weight, not market value.
- **DAIO integration:** BONA balances feed into the Voting Citizen rotation logic (Seats 8-12). Senatus seat assignments are weighted by BONA over a 90-day rolling window. The rotation is processed by Curia at the start of each governance epoch.

Section 7 — Mainnet Deployment Sequence / Runbook

Pre-flight (1 week before):

1. AlgoKit 3.0 LocalNet + Foundry Anvil parity tests for the full system. All `pytest` and `forge test` suites green.

2. Run an external audit on `Genius`, `Forum`, `Curia`, `Sponsio Pactum`, and `XPythai` (Tinyman v2 architectural overlap means audit reviewers can lean on the published Tinyman Runtime Verification work as a baseline).
3. Notarize the audit report hashes via a small Algorand txn (note field = sha256 of audit PDF).

Algorand deployment (in order, every step idempotent via `algokit deploy`):

1. Deploy `Fides` (Pyth adapter), seed with treasury_assets list.
2. Deploy `Tabularium` referencing Fides.
3. Mint the 13 Senatus ASAs (one atomic group; `note` field on each = its seat number 0-12).
4. Deploy `Curia` referencing the Senatus collection.
5. Deploy `Sponsio Pactum` and `Censura` referencing Curia.
6. Deploy `Genius`.
7. Deploy `Forum.validator_app` with `GENIUS_APP_ID` baked into approval program.
8. Deploy `ntt_token` (locking mode, references canonical PYTHAI ASA), `ntt_rate_limiter`, `ntt_manager`, `transceiver_manager` (Folks Finance Algorand NTT stack).
9. Deploy `state_proof_emitter` for governance bridging.
10. Run `bootstrap` + `add_initial_liquidity` for the genesis pool (PYTHAI/USDC). This is the moment the 100-day timelock begins.
11. Renounce all deployer keys: each app's approval program contains a one-time `renounce()` call that wipes the configurable global state slot holding the deployer address. Execute and verify.
12. Notarize all app IDs into `allchain.html` and sign-commit.

Ethereum mainnet deployment (Foundry, deploy at <0.2 gwei window):

1. `forge create XPythai.sol` with constructor setting per-bridge limits.
2. `forge create XLockbox.sol`.
3. Deploy `LZv2Adapter`, `CCIPAdapter`, `WormholeAdapter`. Wire each to XPythai with `XPythai.setLimits(...)`.
4. `forge create EVMVoteSink.sol` with the Falcon verifier address.
5. Call `XPythai.freezeLimits()` (the one-shot `LIMITS_FROZEN` toggle). After this, no rate limit can ever be changed.
6. Notarize EVM addresses into `allchain.html`, sign-commit.

Post-deploy verification:

- All Algorand apps have no `manager`; all EVM contracts return `address(0)` from `owner()`.
- A read-only call to `Genius.state(genesis_pool)` returns `LOCKED_EPOCH` with `lock_start_ts ≈ deploy_block_time`.
- `XPythai.LIMITS_FROZEN()` returns true.
- A test x402 payment through `https://x402-avm.pythai.net/v1` succeeds for a 0.001 USDC test invoice.
- The `allchain.html` SHA256 is published on the PYTHAI Twitter for community pinning.

Operational stance post-deploy:

- OpenBSD vmm guests run rootless Podman containers for: x402-avm facilitator (PHP-FPM), mindX, BANKON, AgenticPlace.
 - No service holds any deployer private key. Genius redemption proceeds purely from user-signed atomic groups, so PYTHAI ops cannot accelerate, delay, or block a redemption.
 - Monitoring: every `Genius` state transition, every Tessera-LP mint, every cross-chain bridge call, every x402 facilitator settle is alerting-eligible. NiceGUI dashboard pulls indexer.
-

Recommendations

Stage 0 (this week) — Lock the BONAFIDE naming into a single repository

(`github.com/pythaiml/bonafide`) with the contract stubs, this document as README, and a CC0-licensed glossary of Latin terms. This claims the namespace publicly before anyone else uses these names.

Stage 1 (next 30 days) — Build `Genius` and the modified `Forum.validator_app` first, end-to-end in AlgoKit LocalNet with full `algorand-python-testing` coverage. Do not touch governance yet. The timelock is the single most important new invariant; prove it first.

Stage 2 (days 30-60) — Build `Senatus`, `Curia`, `Sponsio Pactum`, `Censura`, `Tabularium`, `Fides`. Test against Algorand testnet. Wire the mindX API surface (`/v1/dao/*`, `/v1/oracle/*`).

Stage 3 (days 60-90) — Build the EVM side: `XPythai`, `XLockbox`, three bridge adapters, `EVMVoteSink` with the Falcon verifier. Foundry forge-test against an Ethereum mainnet fork. Aim for the <0.2 gwei deploy window.

Stage 4 (days 90-120) — Build `x402_avm` scheme, publish it to the x402 specs repo as an open PR (this is a community contribution and earns ecosystem credibility). Ship the PHP middleware on mindX.

Stage 5 — Genesis pairing. Run the deployment runbook (§7). This is the moment the 100-day timelock starts. Public announcement should explicitly state that the auto-roll makes "wait for expiry then rug" impossible, to forestall the "100 days is too short" criticism that DEXTools/Phemex baselines would otherwise trigger.

Benchmarks that change the recommendation:

- If a critical Tinyman v2 vulnerability is disclosed before Stage 1 completes, fork from `tinymanorg/tinyman-amm-contracts-v2` at a pre-vulnerability commit and patch forward; do not blindly take the latest.
 - If LayerZero V2 DVN economics degrade (single-DVN compromises documented in the wild), reduce the security stack threshold and add a second required DVN.
 - If Wormhole NTT on Algorand introduces a Global Accountant or automatic relaying (currently absent per the Folks Finance ImmuneFi scope), revisit `ntt_manager` peer config to opt into that surface.
 - If Algorand state proofs add a Solana or Cosmos verifier, add those EVMVoteSink siblings for non-EVM treasury actions.
-

Caveats

1. **BONAFIDE is first published here.** Research found no prior public artifact for `BONAFIDE`, `Tabularium`, `Fides`, `Sponsio Pactum`, `Censura`, `Senatus`, `Tessera`, `Curia`, `Genius`, or `BonaToken` tied to codephreak. If alternate definitions exist in private repos or chat history, this document supersedes nothing — but it also has no upstream to cite. Treat this document as canonical v1.
2. **100-day timelock clears the explicit red-flag threshold but falls short of the credibility baseline.** DEXTools sets <30 days as a major red flag and 6 months as bare minimum for small-caps; Phemex treats <30 days as a warning sign. 100 days clears the red-flag line comfortably but is materially below the 6-month bar. The auto-roll mechanic structurally addresses this, but the messaging risk is real. `DEXTools` `Phemex`
3. **x402 `exact_avm` is a new scheme.** It will not work with the Coinbase-hosted facilitator on day one. PYTHAI must run its own facilitator (`x402-avm.pythai.net`) and submit the scheme to the x402 community for canonical adoption.
4. **Tessera-LP secondary market introduces redemption-right concentration risk.** Mitigate with the per-redemption pool-depth cap noted in §1.2.
5. **Wormhole NTT on Algorand diverges from EVM/Solana/Sui.** Per the Folks Finance ImmuneFi audit scope, there is no Global Accountant, no additional payload in `NttManager`, and no automatic relaying. Build for this asymmetry; do not assume parity with Solana NTT documentation. `ImmuneFi`

6. **The Folks Finance NTT contracts received a private audit in October 2025 and a public ImmuneFi competition (\$30,000 reward pool) that closed November 20, 2025.** Treat them as recently audited but not battle-tested at the scale of Wormhole's EVM token bridge. The rate-limited xERC20 outer layer is the primary safety net.
7. **All Latin role naming should pass a sanity check with a classicist** before public branding — some terms ("Censura" especially) carry connotations that may not translate well into English-language marketing.
8. **Algorand State Proofs are post-quantum (Falcon) but the consensus layer is still Ed25519.** This is a known limitation of the current Algorand quantum-resistance roadmap. The DAIO is safe against quantum attacks on historical state but not against a quantum attacker on live consensus. This matches the Algorand Foundation's own stated security model. Algorand
9. **The Tinyman V2 protocol-fee model accumulates fees in the underlying pool assets, not in LP tokens** (a change from V1, where 1/6 of swap fees was set aside as protocol-owned LP tokens). The Forum fork preserves this V2 behavior; the genesis-pool revenue model should be planned accordingly — Forum's treasury gets paid in the swapped assets themselves, claimable by any caller's inner-transfer-to-treasury call.